SOFTWARE DESIGN PATTERNS

School Security System

Final project report

A Java application for verified school pickup, single-use QR access and persistent audit reporting.

Team member	Allocated pattern responsibility
Aldiyar Adilkhan	Factory Method and Strategy
Baktiyar Kaiyrzhan	Builder and Observer
Bauyrzhan Muratkhanov	Decorator and Adapter
Zhalgas Sarsenov	Facade and Visitor

Repository: github.com/Buka2989/SecuritySystem - design-patterns

Introduction

School staff need a controlled way to confirm which adult may collect a pupil, authorize an entry and retain a reviewable history. This application links guardians to pupils through administrator review. An approved guardian can issue a time-limited QR; security staff validate it once and export the resulting audit records.

The project extends an existing React and PostgreSQL application with a Java 21 Spring Boot core. Eight application-owned design patterns support configurable behavior, review notifications and report generation. The user journey covers selection, configuration, action, review and export.

This report explains the implemented design, demonstration scenarios, source excerpts, verification evidence and team responsibilities. Individual oral defense and independent work remain the responsibility of each team member.

Main Body

Users and scenarios

Scenario	Inputs and visible outcome
Student administration	ADMIN creates a pupil, changes the class and sets ACTIVE/INACTIVE.
Selectable search	PARENT chooses contains/prefix, enters name and class, selects a match.
Request and review	Guardian chooses relationship type; ADMIN approves or rejects. Rejected requests can be resubmitted; removal cancels pending requests.
Event reaction	After approval commits, the guardian sees a notification and updated relationship status.
QR configuration	Approved guardian selects lifetime and issues or revokes a QR.
Access control	SECURITY scans QR; one use succeeds, repeated/expired/revoked attempts are denied and logged.
Report and persistence	ADMIN/SECURITY filters saved logs, inspects counts and downloads CSV/JSON with optional redaction. Data survives restart.

Architecture and data

React pages call authenticated REST controllers through /api. DTO validation and role checks protect the boundary. Java services enforce lifecycle rules and own transaction boundaries. JPA repositories store six entities in PostgreSQL; Nginx serves the production interface. The UI, business decisions and persistence are separate layers.

User 1..* GuardianStudent *..1 Student; GuardianStudent 1..* VerificationRequest and QrCode; QrCode 1..* AccessLog. Each access log also refers to the security User. Relationship deletion is soft, preserving verification and access history. A named Docker volume retains data across container recreation.

QR verification locks the relationship and QR in a consistent order. QR consumption and the audit insert commit together. A failed audit insert rolls consumption back. Unknown tokens cannot satisfy the audit table foreign key and are logged in application logs instead.

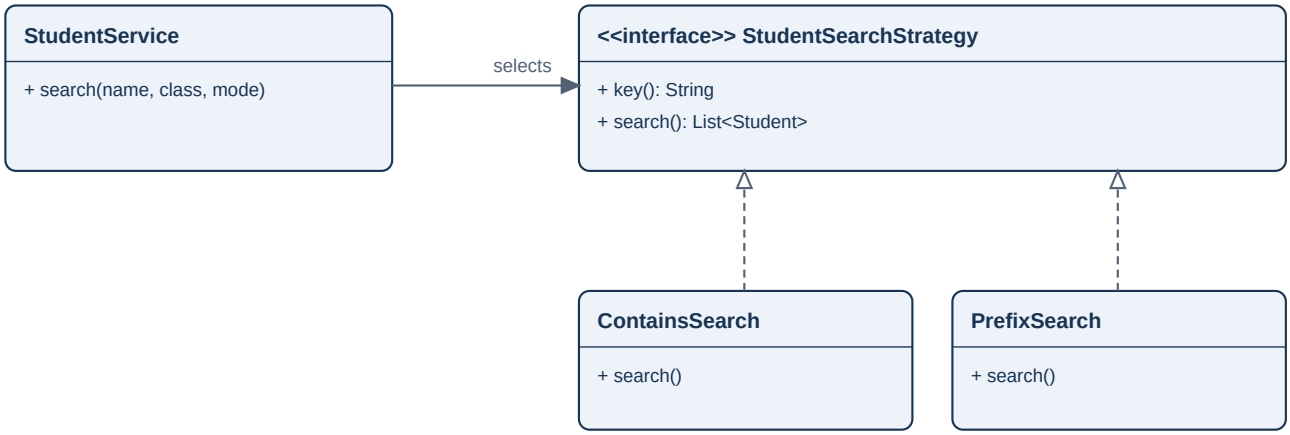
Pattern selection and demonstrations

Pattern / category	Problem and main participants	Demonstration / owner
Builder / creational	QrIssuePlan.Builder collects issue options and validates the immutable QrIssuePlan before issuing QrCode.	Select five-minute QR; inspect expiry. Baktiyar Kaiyrzhan.
Factory Method / creational	ReportExporter defines export; CsvReportExporter and JsonReportExporter override createDocument, returning ReportDocument products.	Download both formats. Aldiyar Adilkhan.
Adapter / structural	AccessLogReportAdapter implements ReportSource to translate existing repositories into typed ReportEntry records.	Scan QR, then inspect report row. Bauyrzhan Muratkhanov.
Decorator / structural	RedactedReportSource wraps ReportSource; produces copied entries with names/IDs hidden, preserving original data.	Toggle redaction; compare exports. Bauyrzhan Muratkhanov.
Facade / structural	AccessReportFacade coordinates date validation, data source, decorator, exporter and visitor behind summary/export operations.	Filter period and export from one page. Zhalgas Sarsenov.
Strategy / behavioral	StudentSearchStrategy is implemented by ContainsSearch and PrefixSearch; StudentService selects the user-requested implementation.	Search Child in Test Child with both modes. Aldiyar Adilkhan.
Observer / behavioral	ReviewPublisher owns ReviewSubscriber registrations. ReviewInbox and ReviewAuditSubscriber react only after a review commits.	Approve in a second session; guardian UI updates. Baktiyar Kaiyrzhan.
Visitor / behavioral	Allowed and Denied entries accept ReportVisitor; StatisticsVisitor counts outcomes and denial reasons through overloaded visits.	First scan + repeat produce total 2, allowed 1, denied 1. Zhalgas Sarsenov.

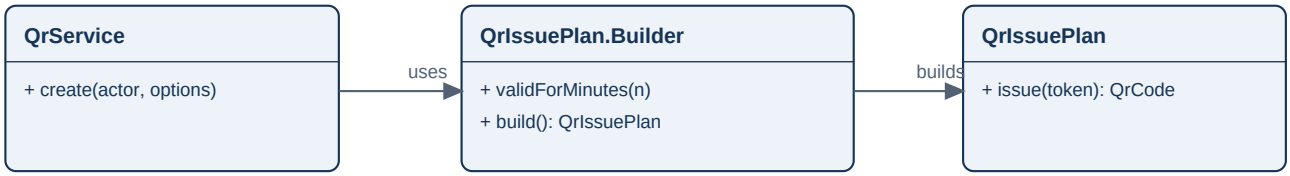
All eight patterns run through the same application. Spring supplies dependency injection and transaction callbacks; the application supplies the pattern participants and interactions. Framework-provided patterns are not counted.

Class diagram for access and review

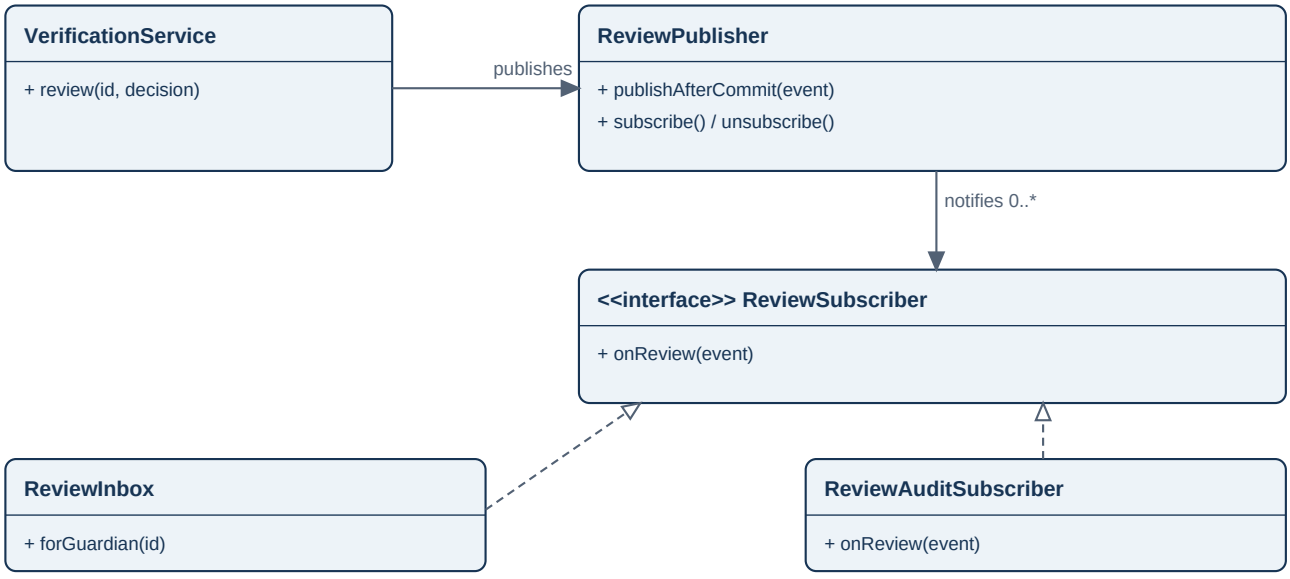
Strategy: user-selected search



Builder: validated QR plan

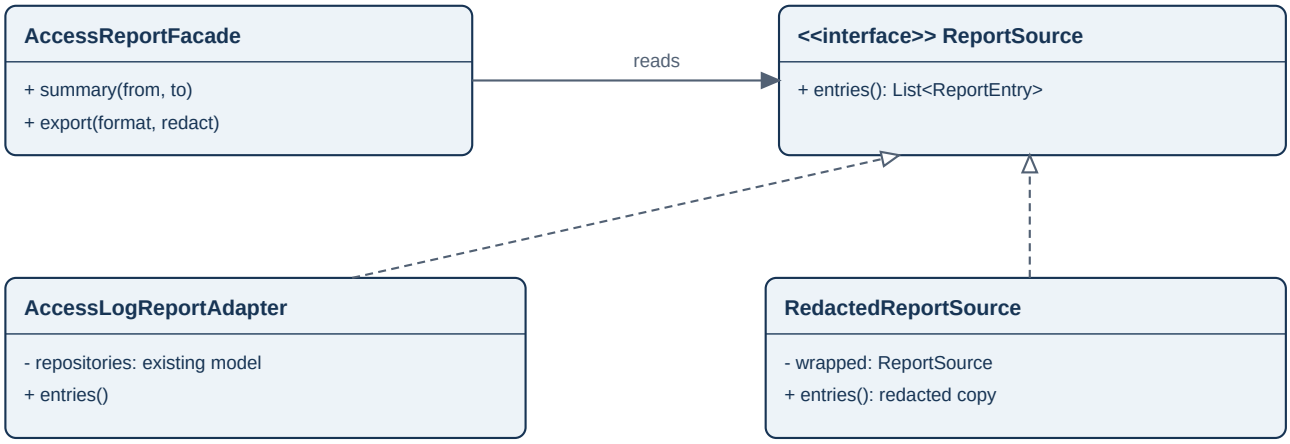


Observer: dispatch after transaction commit



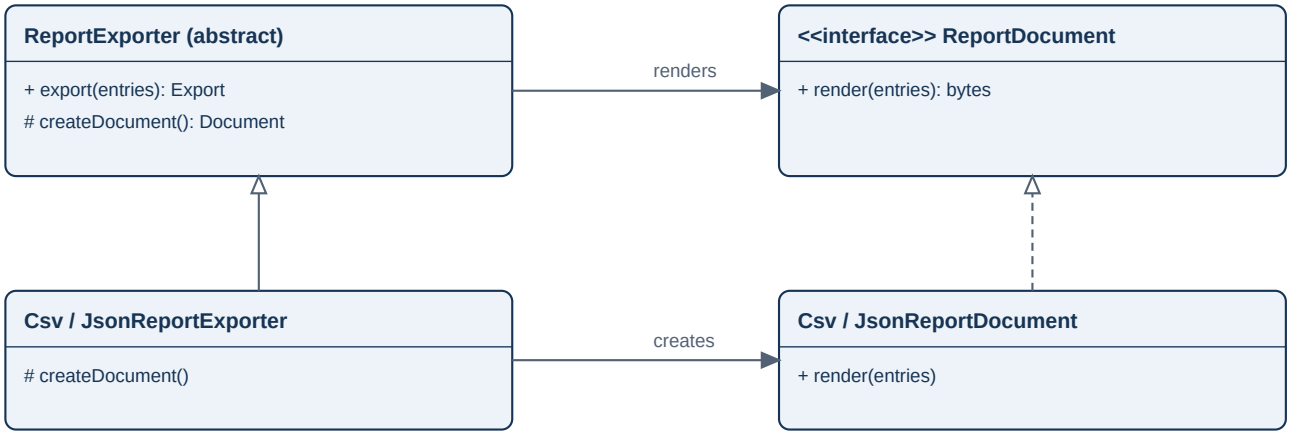
Dashed hollow arrows indicate interface realization. Ordinary arrows indicate use. Concrete search strategies delegate database access to StudentRepository.

Class diagram for reporting

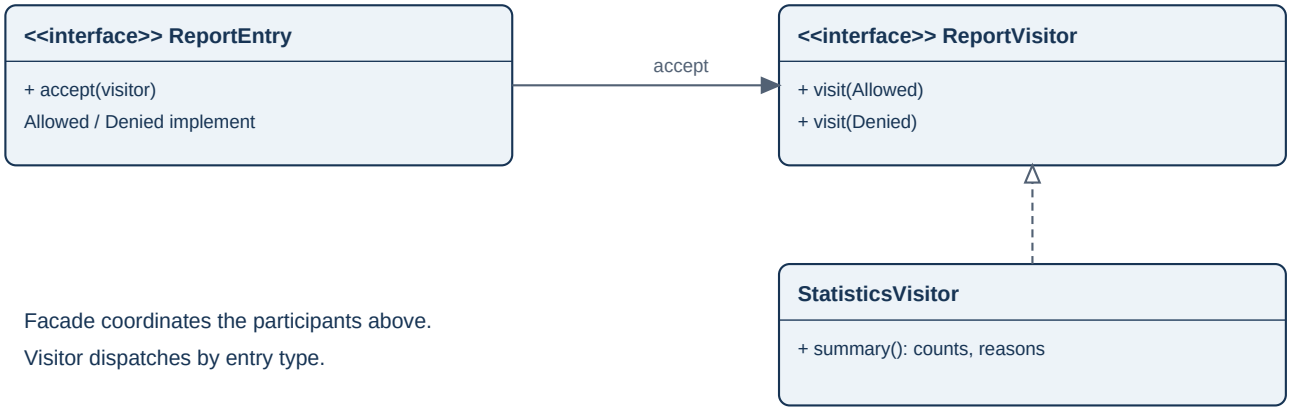


Adapter: relational records to typed entries

Decorator: wraps another ReportSource



Factory Method: each exporter creates its concrete product

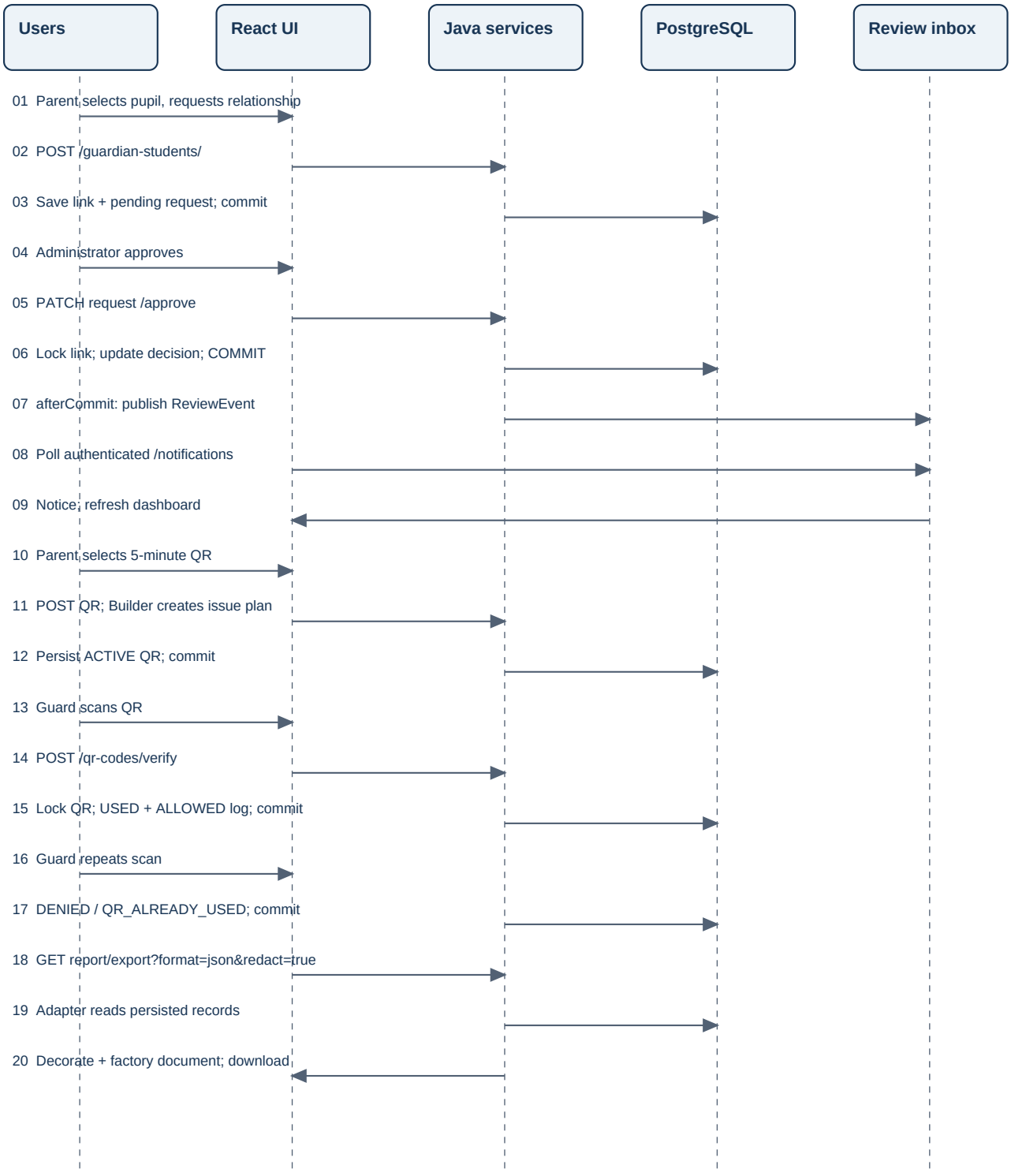


Facade coordinates the participants above.

Visitor dispatches by entry type.

Csv / Json labels abbreviate two corresponding concrete classes, listed in the pattern table. The complete editable class diagram is docs/uml/classes.mmd.

Sequence diagram for the main journey



Each database operation includes a service transaction. The afterCommit callback is essential: failed or rolled-back reviews must not appear as successful notifications. The editable sequence source is docs/uml/journey.mmd.

Creational pattern implementation

Builder prepares a valid QR issue plan

backend/src/main/java/kz/school/security/service/QrService.java

```
QrCode q=kz.school.security.patterns.qr.QrIssuePlan.builder().forRelationship(link
    .id).issuedAt(LocalDateTime.now(clock))
.validForMinutes(data.expiresInMinutes()==null?30:data.expiresInMinutes())
    .build()
.issue(Base64.getUrlEncoder().withoutPadding().encodeToString(bytes));
return DtoMapper.qr(codes.save(q));
```

QrIssuePlan validates that a relationship and issue time exist and the lifetime is between one and 1,440 minutes. The service checks ownership, approval and student activity before construction. Token bytes come from SecureRandom. The plan retains thirty minutes as its default.

Factory Method creates format-specific products

backend/src/main/java/kz/school/security/patterns/report/ReportExporter.java

```
protected abstract ReportDocument createDocument();
public abstract String format();
public final Export export(List<ReportEntry> entries){
ReportDocument document=createDocument();
return new Export(document.render(List.copyOf(entries)), document.contentType(),
    "access-report."+document.extension());
}
```

backend/src/main/java/kz/school/security/patterns/report/JsonReportExporter.java

```
protected ReportDocument createDocument(){return new JsonReportDocument(json);
    }
public String format(){return "json";
    }
```

The inherited export workflow asks the overridden factory method for a product, then uses only ReportDocument operations. Adding a format introduces a product and exporter without rewriting the common workflow. The tradeoff is extra classes for a small number of formats.

Behavioral pattern implementation

Strategy selects the requested algorithm

backend/src/main/java/kz/school/security/service/StudentService.java

```
public List<StudentResponse> search(String name, String className,
    String mode) {
return strategies.stream().filter(s->s.key().equals(mode)).findFirst().orElseThrow(()->new
    ApiException(422, "Unknown search mode"))
.search(name, className).stream().map(DtoMapper::student).toList();
}
```

Observer delivers only committed decisions

backend/src/main/java/kz/school/security/patterns/events/ReviewPublisher.java

```
public void publishAfterCommit(ReviewEvent event) {
if(!TransactionSynchronizationManager.isSynchronizationActive()) throw new
    IllegalStateException("Review publication requires a transaction");
TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization()
    {
@Override public void afterCommit() { dispatch(event);
     }
});
}
```

The publisher isolates subscriber runtime failures. ReviewInbox keeps at most 1,000 deliveries and only returns notices for the authenticated guardian. React polls every three seconds and dispatches verification-updated to refresh the dashboard. A restart removes notices; the underlying verification decision remains persistent.

Visitor dispatches on the concrete entry type

backend/src/main/java/kz/school/security/patterns/report/ReportEntry.java

```
record Allowed(String id, String guardian, String student, String guard,
    String reason, LocalDateTime time) implements ReportEntry {
public void accept(ReportVisitor visitor){visitor.visit(this);
    }
```

StatisticsVisitor implements visit(Allowed) and visit(Denied). Adding another aggregation is independent of report rendering; adding a new entry subtype requires every visitor to handle it.

Structural pattern implementation

Adapter maps the existing model

backend/src/main/java/kz/school/security/patterns/report/AccessLogReportAdapter.java

```
String guardian=users.findById(link.guardianId).orElseThrow().fullName;
String student=students.findById(link.studentId).orElseThrow().fullName;
String guard=users.findById(log.securityUserId).orElseThrow().fullName;
return "ALLOWED".equals(log.result)?(ReportEntry)new ReportEntry.Allowed(log
    .id.toString(), guardian, student, guard, log.reason, log.createdAt)
:new ReportEntry.Denied(log.id.toString(), guardian, student, guard,
    log.reason, log.createdAt);
```

The adapter connects audit rows to their QR, relationship, guardian, pupil and guard. It exposes report entries without altering the inherited schema. The current per-record lookups are adequate for a classroom dataset; larger datasets should use a joined projection and pagination.

Decorator adds a privacy option

backend/src/main/java/kz/school/security/patterns/report/RedactedReportSource.java

```
public class RedactedReportSource implements ReportSource {
private final ReportSource wrapped;
public RedactedReportSource(ReportSource wrapped){this.wrapped=wrapped;
    }
public List<ReportEntry> entries(){return wrapped.entries().stream().map(e->
    e instanceof ReportEntry.Allowed
?(ReportEntry)new ReportEntry.Allowed("",  "Hidden",  "Hidden",  "Hidden",
    e.reason(), e.time())
:new ReportEntry.Denied("",  "Hidden",  "Hidden",  "Hidden", e.reason(),
    e.time())).toList();
    }
}
```

Facade hides orchestration from the controller

backend/src/main/java/kz/school/security/patterns/report/AccessReportFacade.java

```
public StatisticsVisitor.Summary summary(LocalDate from, LocalDate to){
StatisticsVisitor visitor=new StatisticsVisitor();
    entries(from, to, false).forEach(e->e.accept(visitor));
    return visitor.summary();
}
```

The export operation chooses a registered exporter, validates the period and applies optional decoration. Redaction hides selected identifiers and names, while reason codes and timestamps remain. CSV cells are quoted and formula-prefixed values are neutralized before download.

Interface and student workflow

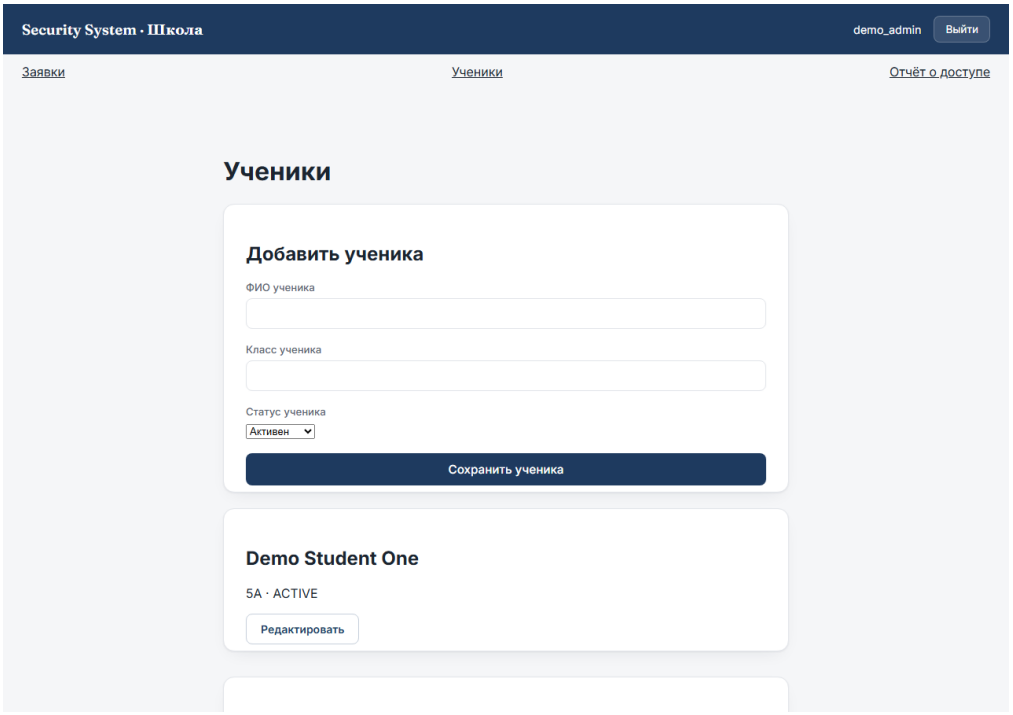


Figure 1. Administrator creates and edits a student through real REST operations.

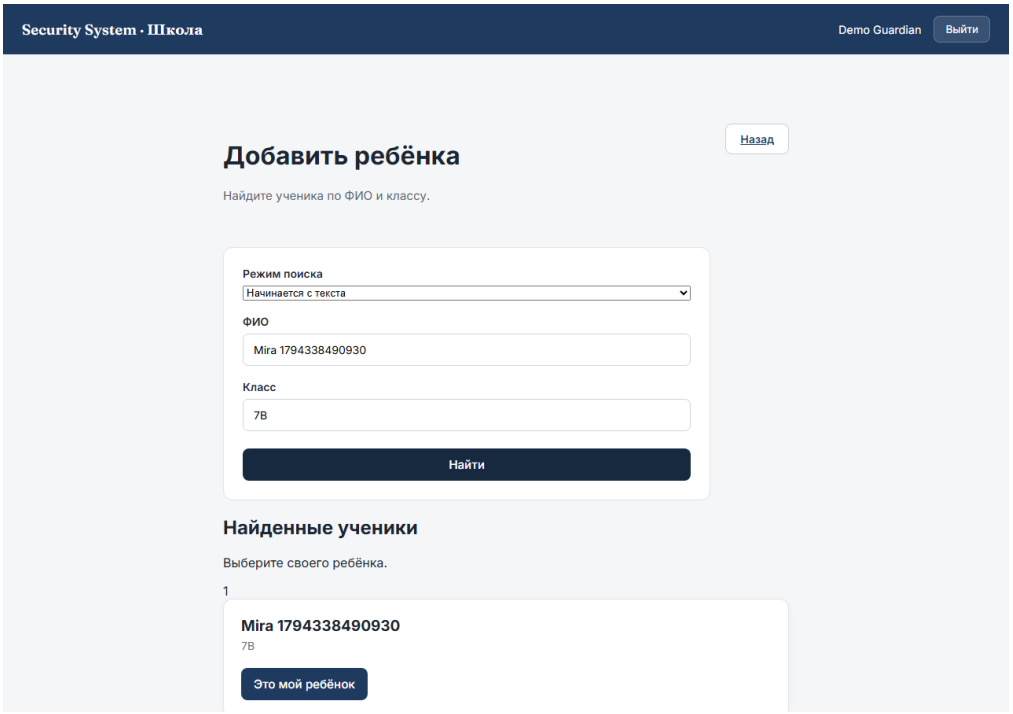


Figure 2. Guardian selects the prefix strategy to narrow student matches.

Review notification and configured QR

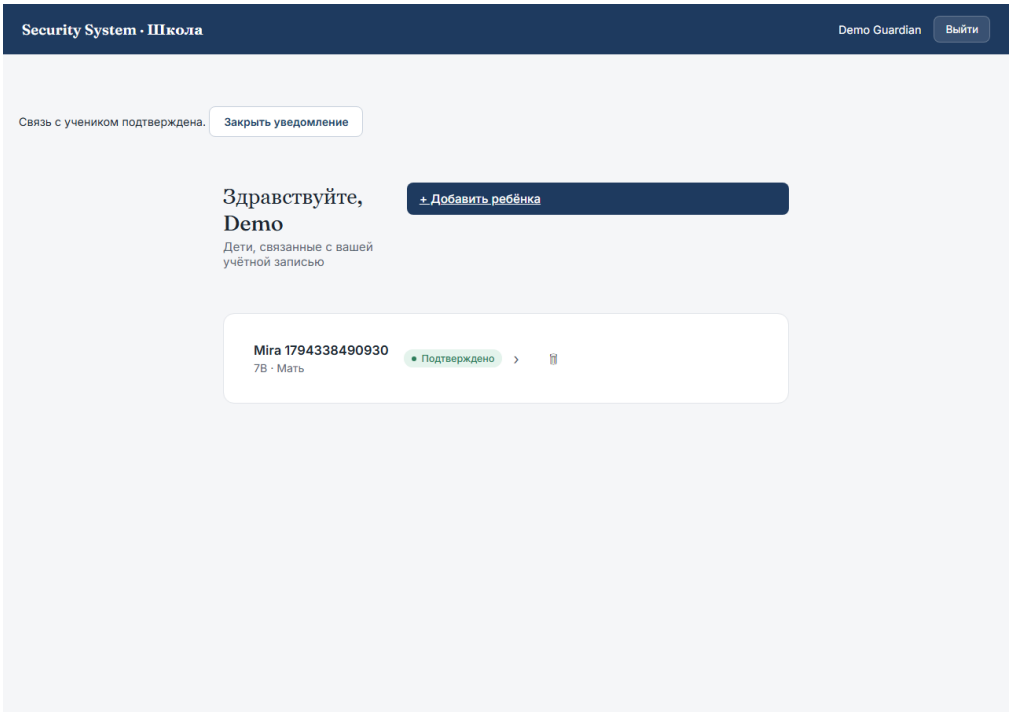


Figure 3. A committed administrator review updates the guardian interface.

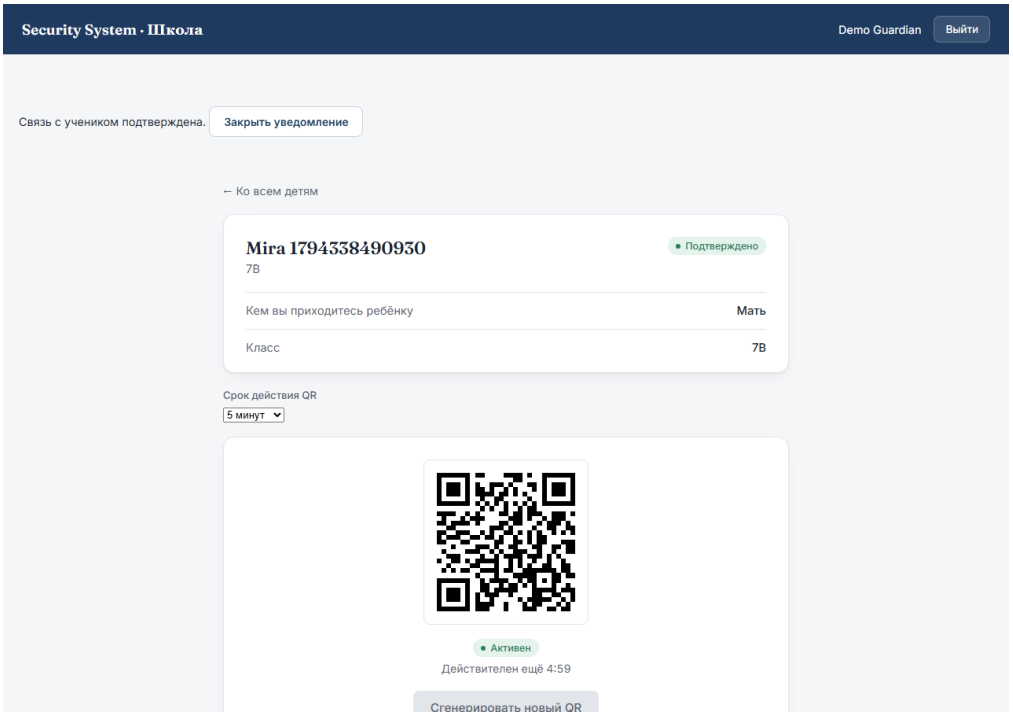


Figure 4. The guardian selects a QR lifetime and receives a timed code.

Reporting and test evidence

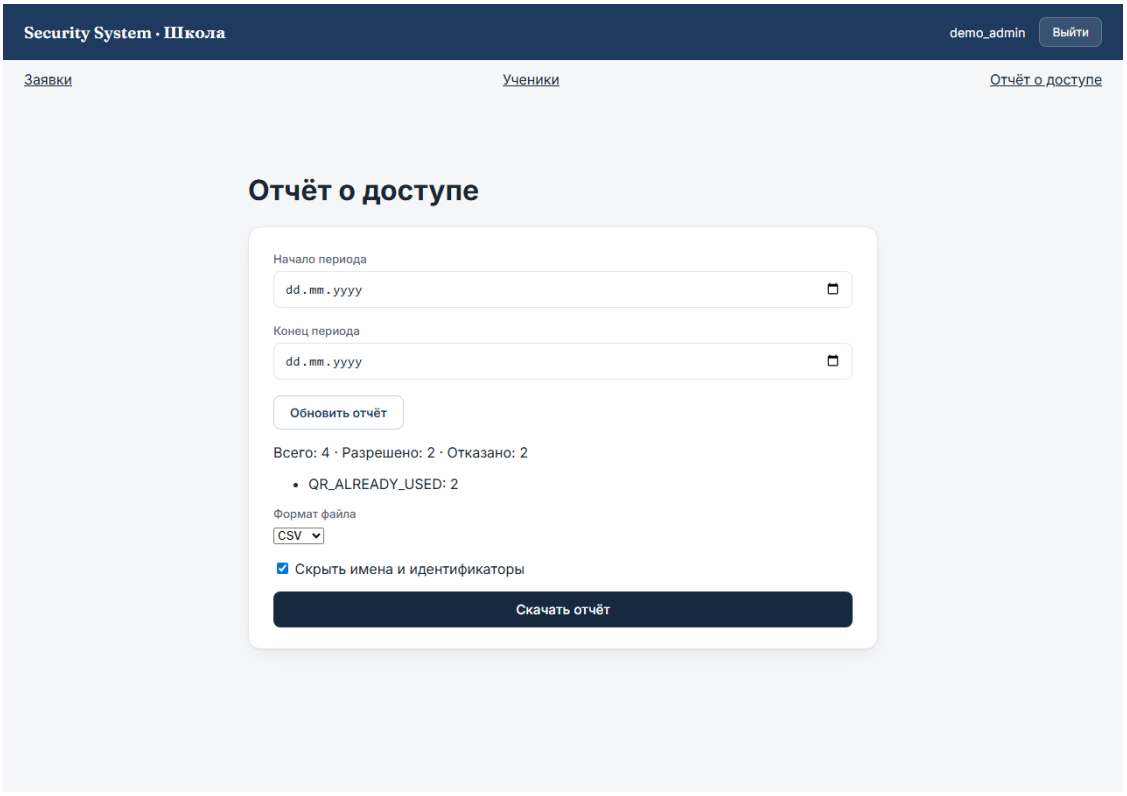


Figure 5. Authorized staff inspect persisted counts and choose export format and redaction.

Verification scope

Verified on: 2026-11-11

Backend: PASS - 24 unit tests and 11 PostgreSQL 18 integration tests; no failures, errors or skipped tests.

Frontend: PASS - TypeScript and Vite 8 production build; npm audit reports zero known vulnerabilities.

Browser: PASS - 2 Playwright Chromium scenarios against the real application, including QR pixel decoding.

Deployment: PASS - PostgreSQL, Java 21 backend and Nginx frontend are healthy in Docker Compose.

Persistence: PASS - counts and complete row checksums match for all six tables after container recreation.

Limitations: Physical camera and HTTPS on a phone were not tested. Notifications are transient; database decisions persist.

The executable evidence is under backend/target/surefire-reports, backend/target/failsafe-reports and e2e/playwright-report after running the suites. CI uploads these reports. The submission archive excludes caches and generated test traces; the scenario matrix below records the relevant results.

Test scenarios and actual results

Automated scenarios exercise real business logic. PostgreSQL API tests cover persistence and transactions; browser tests use the actual interface and endpoints. Inputs, expected results and actual outcomes are recorded below and in docs/TESTING.md.

Scenario and input	Expected result	Actual result
Main journey: request, reject/resubmit, approve, issue QR, scan twice.	First scan ALLOWED; repeated scan DENIED; both saved.	PASS - workflow.spec.js; actual QR pixels decoded.
Search Test Child with Child in contains/prefix modes; edit class.	Contains returns one; prefix zero; edited class reloads.	PASS - ApiIT and patterns.spec.js.
Commit approval; roll back a queued review; read another inbox.	Only owner receives committed notice; rollback is silent.	PASS - Observer unit/API tests and live UI notification.
Recreate all Compose containers without removing volume.	All six tables retain row counts and full content.	PASS - six table checksums match.
Export two or more saved scans as CSV/JSON; toggle redaction.	Valid documents; hidden names/IDs; original data unchanged.	PASS - API, factory/decorator tests and browser downloads.
Issue QR for 5 minutes; try 0 and 1,441 minutes.	Five-minute interval; invalid lifetimes rejected with 422.	PASS - Builder tests and ApiIT.
Blank student name/class; deactivate a linked student.	Blank data rejected with 422; issue/scan disallowed when inactive.	PASS - student editing and inactive-pupil API tests.
Unknown search mode, XML export, reversed date range.	422 and readable errors; inclusive same-day filters work.	PASS - Facade/API tests and browser error assertions.
Parent requests staff reports; another guardian revokes QR.	403 for staff resource; foreign QR is not exposed (404).	PASS - role/ownership API tests.
Eight concurrent scans; four simultaneous resubmissions.	Exactly one ALLOWED and one new pending request.	PASS - concurrent ApiIT scenarios.
Force final access-log insert to violate a database constraint.	409; consumption rolled back to ACTIVE; next scan succeeds.	PASS - transaction rollback ApiIT.

Business logic automation includes JWT and QR policy tests plus Builder, Factory Method, Decorator, Observer, Facade and Visitor tests. Negative cases include invalid lifetime, blank student fields, unsupported search/report modes, reversed dates and unauthorized operations.

Individual contributions and conclusion

Team responsibilities

The team confirmed the allocation shown on the title page. Aldiyar owns search and document creation; Baktiyar owns QR planning and committed event delivery; Bauyrzhan owns data adaptation and privacy decoration; Zhalgas owns report orchestration and aggregation. Each member is responsible for running the relevant tests, demonstrating the two patterns and making a small code change during defense. Detailed exercises and limitations are in docs/DEFENSE.md.

The application inherits the source history of the Java migration and design-pattern extension. Responsibility allocation is not evidence that each student independently authored all allocated code. Weekly progress presentations and individual defense must be completed by the students.

Conclusion

The application supports the complete school-access journey: manage a pupil, configure a guardian link, review it, issue a timed QR, record access and export the saved result. Eight patterns provide distinct extension points within one integrated Java application. The interface is connected to persistent business operations, and automated checks cover successful workflows, alternative behavior, events and invalid actions.

Future Work

Add durable event delivery for multiple backend instances; replace per-row report lookups with joined, paginated queries; introduce deployment-specific HTTPS and camera testing on phones; add account-management and registration screens; and provide retention controls for audit records. These improvements address scale and deployment needs beyond the classroom demonstration.

Tools and external materials

OpenAI Codex assisted with repository analysis, implementation support, regression tests, UML and report preparation. Students must understand, verify and modify the submitted solution in accordance with course policy. No fabricated commit history or independent-authoring claims are used.

External software: Java/OpenJDK, Spring Boot/Security/Data JPA, PostgreSQL, React, TypeScript, Vite, Nginx, Docker Compose, JUnit/AssertJ/Mockito, Testcontainers, Playwright, html5-qrcode, qrcode.react and ReportLab. Dependency versions are pinned or resolved in pom.xml and package-lock.json. Requirements were taken from the supplied FINAL PROJECT document. The README and source repository provide reproducible launch and test commands.